SMART STUDENT TASK MANAGER

A Python-Based Student Productivity Application

PROJECT REPORT

Student	Altaf Bagawan
Branch	Information Science Engineering
Institution	Bapuji Institute of Engineering & Technology, Davanagere
Project Type	Python Mini Project

Academic Project Submission

1. Abstract

Smart Student Task Manager is a lightweight Python application developed to help students organize academic and personal activities efficiently. The system provides a structured interface for creating tasks, assigning priorities, monitoring completion status, deleting tasks and reviewing progress. Task records are stored locally in JSON format, allowing information to persist between application sessions.

2. Introduction

Students regularly handle assignments, laboratory work, examinations, project milestones and other activities. When these tasks are managed through scattered notes, it becomes difficult to identify pending work and monitor progress. This project introduces a simple digital task-management solution that is easy to operate, maintain and extend.

3. Problem Statement

There is a need for a simple student-focused application that can maintain tasks in a centralized location, distinguish important work through priority levels, record completion status and preserve information after the application is closed.

4. Objectives

The main objectives are to develop a practical Python application, provide an easy task-management workflow, implement priority-based organization, maintain persistent local storage and demonstrate core software-development concepts such as functions, data structures, file handling, validation and modular design.

5. Scope of the Project

The current version focuses on individual task management through a command-line interface. It is suitable for assignments, study activities, project work and personal reminders. The architecture can later be extended to support multiple users, databases, notifications and graphical or web interfaces.

6. Technology Stack

Technology	Application
Python 3.9+	Core application development
JSON	Persistent local task storage
pathlib	Cross-platform file handling
datetime	Task creation timestamps

7. Functional Requirements

• The system shall allow users to create new tasks.

• The system shall assign a unique identifier to each task.

• The system shall support Low, Medium and High priority levels.

• The system shall display saved tasks in a structured format.

• The system shall allow users to mark tasks as completed.

• The system shall allow users to delete tasks.

• The system shall display a summary of total, pending and completed tasks.

• The system shall persist task information in a local JSON file.

8. System Modules

Module	Description
Task Creation	Collects task details and stores a new task.
Task Display	Presents task records in a readable table.
Task Completion	Updates the status of a selected task.
Task Deletion	Removes a selected task from storage.
Task Summary	Calculates total, pending and completed tasks.
Data Persistence	Reads and writes task data using JSON.

9. System Workflow

Start → Load saved tasks → Display main menu → Select operation → Validate input → Perform operation → Save updated data → Return to menu → Exit.

10. Data Design

Each task is represented as a JSON object containing an ID, title, priority, status and creation timestamp. All task objects are stored in a JSON array in the local tasks.json file. This approach is lightweight and does not require a separate database server.

11. Advantages

• Simple and user-friendly workflow.

• No third-party libraries are required.

• Works offline after Python is installed.

• Persistent local storage.

• Input validation reduces common entry errors.

• Easy to extend into a larger software system.

12. Limitations

• The current version is command-line based.

• It is designed for a single local user.

• Data is stored only on the local device.

• The current version does not provide automatic notifications.

13. Future Enhancements

Future versions can include a graphical user interface, web/mobile deployment, user authentication, SQLite/MySQL database integration, deadline reminders, recurring tasks, calendar integration, progress

analytics and cloud synchronization.

14. Testing Strategy

Test Case	Expected Result
Add valid task	Task is created with a unique ID.
Empty task title	System rejects the input.
Invalid priority	System asks for a valid priority.
View tasks	All saved tasks are displayed.
Complete valid task	Status changes to Completed.
Delete valid task	Selected task is removed.
Invalid task ID	System reports that the task was not found.
Restart application	Previously saved tasks are loaded.

15. Conclusion

The Smart Student Task Manager successfully demonstrates the development of a practical student-oriented application using Python. It combines modular programming, structured data, file handling, validation and persistent storage into a usable solution. The project also provides a strong foundation for future development towards a database-backed web or mobile productivity platform.

16. Project Files

• **app.py** — complete application source code.

• **requirements.txt** — runtime and dependency information.

• **README.md** — installation, execution and usage documentation.

• **project_report.pdf** — detailed academic project report.